

Latihan 1: Menghitung Total Buku yang Tersedia

Fitur ini menghitung total data buku yang tersedia, termasuk yang sudah difilter melalui fitur pencarian. Total dikelola menggunakan state dan diperbarui otomatis setiap kali terjadi perubahan data.

Code utama:

```
const filteredBooks = useMemo(() => {
  const keyword = search.trim().toLowerCase();
  if (!keyword) return ListBook;
  return ListBook.filter(
    (book) =>
      book.title.toLowerCase().includes(keyword) ||
      book.author.toLowerCase().includes(keyword)
  );
}, [search]);

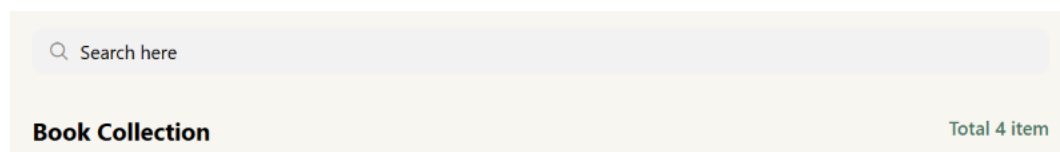
const totalItems = filteredBooks.length;

// Di JSX:
<Text style={{ color: color_list.green, fontWeight: "600" }}>
  Total {totalItems} item
</Text>
```

Alur:

1. Data buku diambil dari ListBook (constants).
2. Saat user mengetik di SearchBar, filteredBooks dihitung ulang via useMemo.
3. totalItems = filteredBooks.length otomatis berubah .
4. Teks header berubah dari "See All" → **"Total x item"**.

Tampilan Dari sisi aplikasi:



Latihan 2: Membuat View "No Record Found"

Penjelasan:

Setiap aplikasi harus menangani kondisi ketika data yang dicari tidak ditemukan. Komponen menampilkan teks **"No record found"** jika hasil filter kosong.

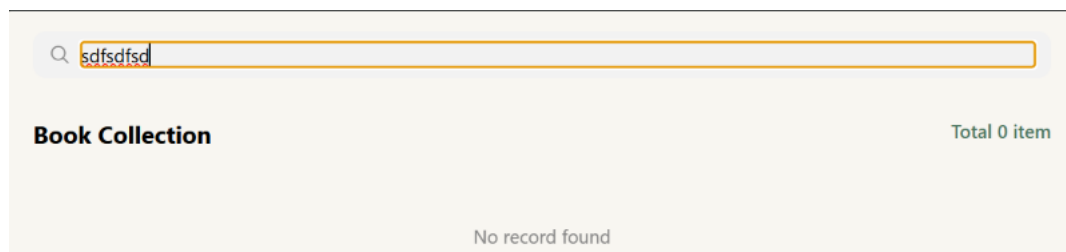
Alur:

1. Setelah filter, cek `totalItems === 0`
2. Jika kosong → render teks "No record found"
3. Jika ada data → render daftar buku seperti biasa

Code: Utama:

```
{totalItems === 0 ? (  
  <Text style={{ textAlign: "center", color: "gray", marginTop: 40 }}>  
    No record found  
  </Text>  
) : (  
  <BookCollection books={pagedBooks} />  
)}
```

Tampilan Dari Sisi Aplikasi:



Latihan 3: Membuat View Paging

Penjelasan:

Fitur pagination membatasi jumlah buku yang ditampilkan per halaman (ITEMS_PER_PAGE = 2). Maksimal 5 nomor halaman ditampilkan. Jika total buku melebihi 100 item, sistem otomatis menampilkan 5 halaman terakhir.

Code Utama:

```
const ITEMS_PER_PAGE = 2;
const MAX_PAGE_BUTTONS = 5;

const totalPages = Math.ceil(totalItems / ITEMS_PER_PAGE);

const pagedBooks = useMemo(() => {
  const start = (currentPage - 1) * ITEMS_PER_PAGE;
  return filteredBooks.slice(start, start + ITEMS_PER_PAGE);
}, [filteredBooks, currentPage]);

const pageNumbers = useMemo(() => {
  if (totalPages <= MAX_PAGE_BUTTONS)
    return Array.from({ length: totalPages }, (_, i) => i + 1);
  if (totalItems > 100) {
    const last = totalPages;
    return [last - 4, last - 3, last - 2, last - 1, last];
  }
  let start = Math.max(1, currentPage - 2);
  let end = Math.min(totalPages, start + MAX_PAGE_BUTTONS - 1);
  return Array.from({ length: end - start + 1 }, (_, i) => start + i);
}, [totalPages, currentPage, totalItems]);
```

Alur:

1. `totalPages = Math.ceil(totalItems / ITEMS_PER_PAGE)`
2. `pagedBooks = slice data sesuai halaman aktif`
3. Tombol Prev/Next untuk navigasi
4. Jika `totalItems > 100` → tampilkan 5 halaman terakhir
5. Saat search berubah → `currentPage` reset ke 1

Tampilan dari sisi Aplikasi:

